Back of Envelope Calculation in System Design Real World Examples

systemdesignschool.io/fundamentals/back-of-envelope-calculation-real-world-examples

Back-of-the-envelope Resource Estimation Real World Examples

Now let's take a look at a few examples of how different systems have different bottlenecks and how we can optimize them. Don't worry if the technologies mentioned in the suggestions sound intimidating. We will cover how to use them (and even have hands-on labs!) in later sections.

1. E-commerce website

- Read-to-write ratio: 10:1
- Throughput: Moderate to high read throughput and low to moderate write throughput.
- Latency: Low latency for product browsing and search, moderate latency for order processing.
- Access pattern: Frequent reads for product information, occasional writes for orders, user profile updates, and product catalog updates.
- Consistency: Strong consistency is important for inventory and order management to prevent overselling, while eventual consistency can be acceptable for product catalog information.

Suggested technologies:

- Caching: Redis or Memcached to cache frequently accessed product information and reduce load on the database.
- Database: Use a database like Amazon Aurora or Google Cloud Spanner, which can handle both read-heavy and write-heavy workloads while providing strong consistency for inventory and order management.

- Read replicas: Implement read replicas to scale out read operations and distribute the read load, while maintaining strong consistency.

2. Social media platform

- Read-to-write ratio: 3:1
- Throughput: High read and write throughput.
- Latency: Low latency for content delivery and user interactions.
- Access pattern: Frequent reads for user feeds, posts, and profiles; frequent writes for new posts, comments, and likes.
- Consistency: Eventual consistency is generally acceptable for social media platforms, as minor delays in displaying new posts, comments, or likes do not significantly impact the user experience.

Suggested technologies:

- Caching: Redis or Memcached for caching user feeds and frequently accessed data.
- Database: Apache Cassandra or Amazon DynamoDB for handling high write throughput, partitioning data effectively, and providing eventual consistency.
- Message queues: RabbitMQ or Apache Kafka for decoupling write operations from other parts of the system, providing asynchronous processing, and managing user feeds.

3. Analytics platform

- Read-to-write ratio: 1:100
- Throughput: Low read throughput and very high write throughput.
- Latency: Moderate to high latency is acceptable for data processing and analysis.
- Access pattern: Infrequent reads for data analysis and visualization; very frequent writes for ingesting and processing large volumes of data.
- Consistency: Eventual consistency is generally acceptable for analytics platforms, as the primary goal is to process and store large volumes of data for later analysis.

Suggested technologies:

- Database: Apache Cassandra, Amazon DynamoDB, or Apache HBase for handling high write throughput, partitioning data effectively, and providing eventual consistency.
- Data processing: Apache Flink, Apache Kafka Streams, or Apache Beam for real-time data processing and stream processing.
- Data warehousing: Google BigQuery, Amazon Redshift, or Snowflake for storing and analyzing large amounts of structured and semi-structured data.

4. Ticket booking system

- Read-to-write ratio: 5:1
- Throughput: Moderate read throughput and low to moderate write throughput.
- Latency: Low latency for browsing events and booking tickets.

- Access pattern: Frequent reads for event and venue information; occasional writes for booking and order management.
- Consistency: Strong consistency is crucial for ticket booking systems to prevent overbooking or selling the same seat to multiple customers.

Suggested technologies:

- Caching: Redis or Memcached for caching frequently accessed event and venue information.
- Database: Use a database like PostgreSQL, Amazon Aurora, or Google Cloud Spanner, which provides strong consistency for ticket inventory and booking management.
- Transaction management: Implement transaction management and optimistic concurrency control to handle concurrent bookings and ensure consistency in the booking process.
- Message queues: RabbitMQ or Apache Kafka.

5. Chat application like WhatsApp

- Read-to-write ratio: 2:1
- Throughput: High read and write throughput for message delivery.
- Latency: Low latency for real-time messaging and user interactions.
- Access pattern: Frequent reads for message retrieval and user status updates; frequent writes for sending messages, updating read receipts, and user presence.
- Consistency: Eventual consistency is usually acceptable for chat applications, as small delays in message delivery do not significantly impact the user experience. However, strong consistency may be required for features such as read receipts and message deletion.

Suggested technologies:

- Caching: Redis or Memcached for caching user profiles, contact lists, and group information.
- Database: Apache Cassandra or Amazon DynamoDB for handling high write throughput, partitioning data effectively, and providing eventual consistency for message storage.
- Message queues: RabbitMQ or Apache Kafka for decoupling message sending and receiving operations from other parts of the system, providing asynchronous processing, and managing message delivery.
- WebSockets: Use technologies like Socket.IO or WebSocket API to handle real-time bidirectional communication between the client and the server.
- Load balancing: Use load balancers like HAProxy or Amazon ELB to distribute the load across multiple servers, especially for WebSocket connections, which tend to be the bottleneck for chat applications.

Again, don’t fret if you don’t know how the technologies mentioned here work yet. We have an entire course dedicated to showing you how to do it :)